<!-- SPDX-License-Identifier: GPL-3.0-or-later Copyright (c) 2026 Mohamed Hammad -->

Loran — TODO v0.1

Field	Value
Project	Loran
Document	Task-Level Implementation TODO
Version	0.1.0 (Phase 0 + Phase 1 decomposed; Phase 2/3 stubbed)
Date	2026-05-11
Author	Mohamed Hammad
Maintainer	Mohamed Hammad Mohamed.Hammad@Steelbore.com
Copyright	(c) 2026 Mohamed Hammad
License	GPL-3.0-or-later
Plan	`loran-plan-v0_1.md`
PRD	`loran-prd-v0_1.md`
Spec	`loran-spec-v0_2.md`

Table of Contents

---

1. How to Use This TODO

This document is the operational checklist for building Loran. Each task is sized to fit in 30 minutes to 2 hours of focused work, has a unique ID (`LOR-PXXX-NNN`), and is verifiable when complete.

Working pattern. Pick a work package, work through its task list in order (most have intentional internal dependencies), check tasks off as they land. A WP is complete when every task under it is checked. A phase is complete when every WP under it is complete and the phase's Definition of Done in `loran-plan-v0_1.md` §13 passes.

Grep for open tasks. `rg '^- \[ \]' TODO.md` returns every unchecked task. `rg '^- \[ \] \*\*LOR-P001'` returns every open Phase 1 task. `rg 'LOR-P000-005'` jumps to a specific task.

This document is mutable. Add tasks during implementation if they emerge. Keep IDs sequential within a phase — don't renumber, even if a task is later deleted (leave the ID retired).

When Phase 1 is tagged, this TODO is revised to v0.2 with Phase 2 decomposed. Phase 3 follows at the start of Billet completion. This keeps the document focused on near-term work.

---

2. Task ID Scheme

`LOR-PXXX-NNN` where:

- `LOR` — Loran project prefix (matches Steelbore convention)
- `PXXX` — three-digit phase number (`P000` = Pre-Phase, `P001` = Ingot, `P002` = Billet, `P003` = Bloom)
- `NNN` — three-digit sequential task number within the phase (zero-padded)

WPs are grouping headers (e.g., `WP-P0.01`, `WP-P1.05`) and do not appear in task IDs. Task numbers are sequential **across the whole phase**, not reset per WP — this makes IDs stable when WPs are reordered.

Cross-cutting workstream tasks use `LOR-PCC-NNN` (rare in this document; most cross-cutting work lands in Phase 0).

---

3. Progress Summary

Phase	WPs	Tasks (this revision)	Status
Phase 0 — Setup	5	41	Not started
Phase 1 — Ingot	19	146	Not started
Phase 2 — Billet	18	*(stubs)*	Deferred to v0.2

Phase	WPs	Tasks (this revision)	Status
Phase 3 — Bloom	7	*(stubs)*	Deferred to v0.3
Total	49	187	

Update this table whenever the document is revised.

---

4. Phase 0 — Pre-Phase Setup (Detailed)

Phase outcome: A repo with posture files, agent context, workspace skeleton, and bootstrap CI. The next commit after Phase 0 closes can be the first Phase 1 task.

WP-P0.01 — Repository initialisation

Sizing: XS | **Critical Path:** Yes | **Plan §:** 4 | **Deps:** —

- ☐ **LOR-P000-001** — Initialise git repository with `git init`
- ☐ **LOR-P000-002** — Add `LICENSE` containing verbatim GPL-3.0-or-later text
- ☐ **LOR-P000-003** — Add `.gitignore` excluding `target/`, `*.bak`, `.DS_Store`, `.idea/`, `.vscode/`, `node_modules/`, `*.swp`
- ☐ **LOR-P000-004** — Add `.editorconfig` (UTF-8, LF endings, 4-space Rust, 2-space TOML/Markdown/YAML)
- ☐ **LOR-P000-005** — Initial commit with DCO sign-off (`git commit -s`)

WP-P0.02 — Posture files

Sizing: S | **Critical Path:** Yes | **Plan §:** 4 | **Deps:** WP-P0.01

- ☐ **LOR-P000-006** — Create `README.md` skeleton: title, tagline, badges placeholder, section headings (Overview, Installation, Quickstart, Project Posture, Maintainer)
- ☐ **LOR-P000-007** — Author README "Project Posture" section per Standard §5.1 (personal hobby, no SLA, GPL-3.0-or-later)
- ☐ **LOR-P000-008** — Author README "Maintainer" section per Standard §13.2 (name, email, project URL, copyright year)
- ☐ **LOR-P000-009** — Add README "Overview" with the three-questions framing from PRD §1
- ☐ **LOR-P000-010** — Author `NOTICE.md` with no-warranty / no-liability statement deferring to GPL-3.0-or-later
- ☐ **LOR-P000-011** — Author `CONTRIBUTING.md`: PR scope, DCO sign-off requirement, security-reporting path, license-of-contributions, maintainer-discretion (Standard §5.4)
- ☐ **LOR-P000-012** — Cross-reference README → NOTICE, CONTRIBUTING, LICENSE via links

WP-P0.03 — Agent context files

Sizing: XS | **Critical Path:** No | **Plan §:** 4 | **Deps:** WP-P0.02

- [ ] **LOR-P000-013** — Author `AGENTS.md` listing every coding invariant: Rust-only, GPL-3.0-or-later + SPDX, run `cargo fmt && cargo clippy -- -D warnings && cargo test` before commit, no `unsafe` outside review, Nushell/Ion/POSIX-sh shells only (no Bash-isms), conventional-commits, DCO sign-off
- [ ] **LOR-P000-014** — Author `CLAUDE.md` referencing the four governing skills (`steelbore-standard`, `steelbore-cli-standard`, `steelbore-agentic-cli`, `rust-guidelines`) and the three governing documents (spec, PRD, plan)
- [ ] **LOR-P000-015** — Author `SKILL.md` with Loran capability-surface metadata for the Steelbore Skills system

WP-P0.04 — Cargo workspace skeleton

Sizing: S | **Critical Path:** Yes | **Plan §:** 4 | **Deps:** WP-P0.01

- [ ] **LOR-P000-016** — Create workspace root `Cargo.toml` with `[workspace]` section listing 9 members (8 crates + `xtask`) and SPDX header
- [ ] **LOR-P000-017** — Add `[workspace.dependencies]` table to root `Cargo.toml` with `serde`, `serde_json`, `toml`, `thiserror`, `anyhow`, `tracing`, `tracing-subscriber`, `jiff`, `clap`
- [ ] **LOR-P000-018** — Create `rust-toolchain.toml` pinning to latest stable Rust (whatever it is at start of work)
- [ ] **LOR-P000-019** — Create `rustfmt.toml` with `imports_granularity = "Crate"` and edition = 2024 (or current)
- [ ] **LOR-P000-020** — Create `.clippy.toml` with project-specific lint config (e.g., `disallowed-methods = []` placeholder)
- [ ] **LOR-P000-021** — Create `crates/loran-cli/` with `Cargo.toml`, `src/main.rs` (placeholder `fn main() { println!("loran") }`), SPDX headers
- [ ] **LOR-P000-022** — Create `crates/loran-core/` with `Cargo.toml`, `src/lib.rs` (`pub fn placeholder() {}`), SPDX headers, `#![forbid(unsafe_code)]`
- [ ] **LOR-P000-023** — Create `crates/loran-index/` with `Cargo.toml`, `src/lib.rs`, SPDX headers, `#![deny(unsafe_code)]`
- [ ] **LOR-P000-024** — Create `crates/loran-pages/` with `Cargo.toml`, `src/lib.rs`, SPDX headers, `#![forbid(unsafe_code)]`
- [ ] **LOR-P000-025** — Create `crates/loran-render/` with `Cargo.toml`, `src/lib.rs`, SPDX headers, `#![forbid(unsafe_code)]`
- [ ] **LOR-P000-026** — Create `crates/loran-tldr/` with `Cargo.toml`, `src/lib.rs`, SPDX headers, `#![deny(unsafe_code)]`
- [ ] **LOR-P000-027** — Create `crates/loran-tui/` with `Cargo.toml`, `src/lib.rs`, SPDX headers, `#![deny(unsafe_code)]`

☐ **LOR-P000-028** — Create `crates/loran-mcp/` with `Cargo.toml`, `src/lib.rs`, SPDX headers, `#![deny(unsafe_code)]`
☐ **LOR-P000-029** — Create `xtask/` with `Cargo.toml`, `src/main.rs` (placeholder), SPDX headers
☐ **LOR-P000-030** — Verify `cargo build --workspace` succeeds with zero warnings
☐ **LOR-P000-031** — Write a small shell script (POSIX `sh`) or `xtask` command that greps for SPDX headers in every `.rs` and `Cargo.toml`; verify it passes

WP-P0.05 — Bootstrap CI pipeline

Sizing: M | **Critical Path:** No (but blocks confident merging) | **Plan §:** 4 | **Deps:** WP-P0.04

☐ **LOR-P000-032** — Create CI workflow file (`.github/workflows/ci.yml` or equivalent for chosen forge)
☐ **LOR-P000-033** — Add `cargo fmt --check` job step
☐ **LOR-P000-034** — Add `cargo clippy --workspace --all-targets -- -D warnings` job step
☐ **LOR-P000-035** — Add `cargo test --workspace` job step
☐ **LOR-P000-036** — Add `cargo audit` job step (install `cargo-audit` if not pre-cached)
☐ **LOR-P000-037** — Add SPDX-header check as a CI step calling the xtask command from LOR-P000-031
☐ **LOR-P000-038** — Configure Tier 1 platform matrix: Linux x86_64 (glibc), Linux x86_64 (musl), Linux aarch64, FreeBSD amd64
☐ **LOR-P000-039** — Configure Tier 2 platform: macOS arm64 with `continue-on-error: true`
☐ **LOR-P000-040** — Configure fail-fast within Tier 1 jobs; Tier 2 jobs report without blocking merge
☐ **LOR-P000-041** — Add CI status badge to top of `README.md`

---

5. Phase 1 — Ingot (Detailed)

Phase outcome (Plan §5): A useful binary that lists, shows, finds, and searches the bundled tool catalog, with full JSON output and SFRS-compliant flags. No network, no overlays, no TUI.

WP-P1.01 — Page parser (`loran-pages`)

Sizing: M | **Critical Path:** Yes | **Plan §:** 5 | **Deps:** WP-P0.04 | **PRD:** FR-035, FR-036, FR-080

☐ **LOR-P001-001** — Add `serde`, `serde_derive`, `toml`, `thiserror` to `loran-pages/Cargo.toml` (use workspace dependencies)
☐ **LOR-P001-002** — Define `Page` struct in `loran-pages/src/page.rs` mirroring spec §6.1 schema fields
☐ **LOR-P001-003** — Implement `Deserialize` for the frontmatter struct via `serde` derive

- [ ] **LOR-P001-004** — Implement frontmatter splitter: locate the `+++` fences, return (frontmatter_str, body_str)
- [ ] **LOR-P001-005** — Implement TOML frontmatter deserialisation into the struct
- [ ] **LOR-P001-006** — Implement required-field presence check (`name`, `category`, `summary`)
- [ ] **LOR-P001-007** — Implement `summary` length validation (⩽120 characters)
- [ ] **LOR-P001-008** — Implement `safe_alias_for ⊆ replaces` invariant check; error includes the offending name
- [ ] **LOR-P001-009** — Implement category-name well-formedness check (slash-tolerant; no leading/trailing slash; no double-slash)
- [ ] **LOR-P001-010** — Define `PageError` enum with `thiserror`: `MissingFrontmatter`, `InvalidToml`, `MissingField(name)`, `SummaryTooLong`, `InvalidSafeAliasFor(name)`, `InvalidCategory(name)` and others as needed
- [ ] **LOR-P001-011** — Implement `Page::parse(input: &str) -> Result<Page, PageError>` as the public entry point
- [ ] **LOR-P001-012** — Add unit tests for 6+ valid pages (minimum viable, full schema, slash-category, multi-replaces, with-pairs-with, with-safe-alias-for)
- [ ] **LOR-P001-013** — Add unit tests for each `PageError` variant (one test per failure mode)
- [ ] **LOR-P001-014** — Add rustdoc comments to `Page`, `PageError`, `Page::parse`, and all public fields

WP-P1.02 — Index loader + `Ingestor` trait (`loran-index`)

Sizing: M | **Critical Path:** Yes | **Plan §:** 5 | **Deps:** WP-P1.01 | **PRD:** FR-070

- [ ] **LOR-P001-015** — Add `loran-pages` as a path dependency in `loran-index/Cargo.toml`
- [ ] **LOR-P001-016** — Define the `Ingestor` trait with `fn ingest(&self) -> Result<Vec<Page>, IngestError>` in `loran-index/src/ingestor.rs`
- [ ] **LOR-P001-017** — Define `IngestError` enum (`Io`, `Page(PageError)`, `BadSource`)
- [ ] **LOR-P001-018** — Implement `MarkdownPagesIngestor` that takes a root directory and walks for `*.md`
- [ ] **LOR-P001-019** — In `MarkdownPagesIngestor::ingest`, parse every found file via `loran-pages::Page::parse` and fail-loud on any error (no silent skipping per FR-035)
- [ ] **LOR-P001-020** — Define `Index` struct holding primary `BTreeMap<String, Page>` keyed by `name`
- [ ] **LOR-P001-021** — Add secondary indexes to `Index`: by-category, by-replaces (multimap), by-tag (multimap)
- [ ] **LOR-P001-022** — Implement `Index::build(pages: Vec<Page>) -> Result<Index, IndexError>` with duplicate-name detection
- [ ] **LOR-P001-023** — Implement lookup methods: `Index::get(name)`, `Index::by_category(cat)`, `Index::by_replaces(name)`, `Index::all()`
- [ ] **LOR-P001-024** — Add unit tests for `Ingestor` (mock impl + verify trait constraints)
- [ ] **LOR-P001-025** — Add unit tests for `MarkdownPagesIngestor` with a fixture directory tree

- [ ] **LOR-P001-026** — Add unit tests for `Index` (build, lookup, duplicate-detection)
- [ ] **LOR-P001-027** — Add rustdoc to `Ingestor`, `MarkdownPagesIngestor`, `Index`, and all public methods

WP-P1.03 — Bundled pages tree (build-time)

Sizing: M | **Critical Path:** Yes | **Plan §:** 5 | **Deps:** WP-P1.01, WP-P1.02 | **PRD:** M-01, M-02

- [ ] **LOR-P001-028** — Create top-level `pages/` directory and `pages/categories.toml` with initial categories: `file-listing`, `file-viewing`, `text-search`, `file-search`, `process-management`, `system-monitoring`, `networking`, `version-control`, `shell-utilities`, `data-processing`
- [ ] **LOR-P001-029** — Author 5 curated pages for file-listing / file-viewing / text-search core trio: `eza.md`, `bat.md`, `rg.md`, `fd.md`, `dust.md`
- [ ] **LOR-P001-030** — Author 5 curated pages for process / system tools: `procs.md`, `bottom.md`, `gping.md`, `hyperfine.md`, `xcp.md`
- [ ] **LOR-P001-031** — Author 5 curated pages for network / shell tools: `dog.md`, `xh.md`, `jaq.md`, `sd.md`, `gitui.md`
- [ ] **LOR-P001-032** — Author 5 curated pages for shell / editor tools: `nushell.md`, `ion.md`, `helix.md`, `zellij.md`, `starship.md`
- [ ] **LOR-P001-033** — Author 5+ additional pages covering the remaining 10 most-relevant tools to bring total to 25+ (`tealdeer`, `lsd`, `delta`, `tokei`, `bandwhich`, others per maintainer's catalog priorities)
- [ ] **LOR-P001-034** — Verify all 20 legacy tools in PRD M-02 are represented in at least one page's `replaces` field
- [ ] **LOR-P001-035** — Verify at least 10 pages include `pairs_with` entries (toward M-03 target of ⩾1.5 average)
- [ ] **LOR-P001-036** — Create `loran-core/build.rs` that reads `pages/` at compile time and emits a generated `BUNDLED_PAGES: &[(&str, &str)]` constant
- [ ] **LOR-P001-037** — In `build.rs`, validate every page via `loran-pages::Page::parse`; fail the build on any error (compile-time validation per Plan §5)
- [ ] **LOR-P001-038** — Implement `BundledPagesIngestor` in `loran-core` that wraps the `BUNDLED_PAGES` constant and implements the `Ingestor` trait

WP-P1.04 — Core resolution chain (`loran-core`)

Sizing: M | **Critical Path:** Yes | **Plan §:** 5 | **Deps:** WP-P1.01, WP-P1.02, WP-P1.03 | **PRD:** FR-010, FR-011, FR-012

- [ ] **LOR-P001-039** — Add `loran-index`, `loran-pages` as path deps in `loran-core/Cargo.toml`
- [ ] **LOR-P001-040** — Add `nucleo-matcher` for fuzzy search
- [ ] **LOR-P001-041** — Define `ShowResult` enum in `loran-core/src/show.rs`: `IndexHit { intro: String, body: ShowBody }`, `NoEntry { hint: String }`

- [ ] **LOR-P001-042** — Define `ShowBody` enum: `Custom { page: Page, source_path: String }`, `None` (tldr variant deferred to Phase 2)
- [ ] **LOR-P001-043** — Implement `resolve_show(index: &Index, tool: &str) -> ShowResult`
- [ ] **LOR-P001-044** — Construct the no-entry hint string per spec §4.1.1 (includes `loran new <tool> --edit` and `see also: loran search <tool>`)
- [ ] **LOR-P001-045** — Define `FindResult` and `resolve_find(index, query, alias_safe_only: bool) -> FindResult`
- [ ] **LOR-P001-046** — Define `SearchResult` and `resolve_search(index, query) -> SearchResult` using `nucleo-matcher` over name/summary/replaces/tags
- [ ] **LOR-P001-047** — Verify none of the three resolve functions panic on adversarial input (empty string, very long string, Unicode edge cases) — add fuzz-style tests
- [ ] **LOR-P001-048** — Add unit tests for hit/miss/edge cases on all three resolve functions

WP-P1.05 — Live `--help` capture engine (`loran-core`)

Sizing: M | **Critical Path:** No (parallel with index work) | **Plan §:** 5 | **Deps:** WP-P0.04 | **PRD:** FR-020 to FR-025, NFR-034

- [ ] **LOR-P001-049** — Add `which`, `jiff` deps to `loran-core/Cargo.toml`
- [ ] **LOR-P001-050** — Define `HelpResult` struct: `captured_text: String`, `flag_used: HelpFlag (`Help`|`H`|`HelpSub`)`, `captured_at: jiff::Timestamp`, `exit_code: i32`
- [ ] **LOR-P001-051** — Define `HelpError` enum: `BinaryNotFound`, `Timeout`, `SpawnFailed(io::Error)`, `AllFlagsFailed`
- [ ] **LOR-P001-052** — Implement PATH resolution in `loran-core/src/help.rs` using the `which` crate; reject anything that looks like a path (contains `/` or `\\`)
- [ ] **LOR-P001-053** — Implement subprocess spawn with `std::process::Command`, `argv = [tool_path, flag]`, no shell, no `arg("--help &&")` etc.
- [ ] **LOR-P001-054** — Set subprocess env: `PAGER="bat -pp"`, `MANPAGER="bat -pp"`, `LESS=""`; if `which("bat").is_err()`, set `PAGER="cat"` and `MANPAGER="cat"`
- [ ] **LOR-P001-055** — Implement 5-second wall-clock timeout using thread + `try_wait()` polling or via `wait-timeout` crate; on overrun, `child.kill()` and return `HelpError::Timeout`
- [ ] **LOR-P001-056** — Implement retry sequence: try `--help` first; on non-zero exit or empty output, try `-h`; if still bad, try `help` (subcommand); prefer first non-empty success
- [ ] **LOR-P001-057** — Capture stdout + stderr separately; prefer stdout if non-empty, fall back to stderr
- [ ] **LOR-P001-058** — Record `captured_at` as `jiff::Timestamp::now()` and format as `YYYY-MM-DDTHH:MM:SSZ` for output
- [ ] **LOR-P001-059** — Implement `capture_help(tool: &str) -> Result<HelpResult, HelpError>` as the public entry point
- [ ] **LOR-P001-060** — Add unit tests using fixture shell scripts (e.g., `tests/fixtures/echo-help.sh`) that exercise timeout, retry, and capture paths

- [ ] **LOR-P001-061** — Add unit tests that confirm path-traversal rejection (`loran help ../etc/passwd` → `BinaryNotFound`)

WP-P1.06 — Markdown → terminal text renderer (`loran-render`, text mode)

Sizing: S | **Critical Path:** No | **Plan §:** 5 | **Deps:** WP-P1.01 | **PRD:** NFR-050, NFR-051

- [ ] **LOR-P001-062** — Add `pulldown-cmark` to `loran-render/Cargo.toml`
- [ ] **LOR-P001-063** — Implement `render_text(body_md: &str, writer: &mut impl Write) -> io::Result<()>` in `loran-render/src/text.rs`
- [ ] **LOR-P001-064** — Handle `pulldown_cmark::Event` types: `Start/End(Heading)`, `Start/End(Paragraph)`, `Start/End(List)`, `Start/End(Item)`, `Start/End(CodeBlock)`, `Text`, `Code` (inline), `Link`, `Html` (passthrough as text)
- [ ] **LOR-P001-065** — Render headings as plain text with capitalisation; render code blocks indented by 4 spaces; render lists with `-` bullets; render links as `text (url)`
- [ ] **LOR-P001-066** — Ensure no ANSI escape codes appear in output (use a writer wrapper that strips them or simply emit none)
- [ ] **LOR-P001-067** — Add unit tests round-tripping 4+ representative page bodies (paragraph-only, with-headings, with-code, with-list-and-links)
- [ ] **LOR-P001-068** — Verify output passes through `grep '^[A-Z]'` and `awk '{print $1}'` on a sample without error

WP-P1.07 — CLI shell with global flags (`loran-cli`)

Sizing: M | **Critical Path:** No (early-parallel; gates sub-commands) | **Plan §:** 5 | **Deps:** WP-P0.04 | **PRD:** All FR-060 series, NFR-053 to NFR-056

- [ ] **LOR-P001-069** — Add `clap` (derive), `serde`, `serde_json`, `tracing`, `tracing-subscriber`, `jiff`, `anyhow` deps to `loran-cli/Cargo.toml`
- [ ] **LOR-P001-070** — Define top-level `Cli` struct with clap derive containing all SFRS §3 global flags: `--json`, `--format`, `--fields`, `--dry-run`, `--verbose`, `--quiet`, `--no-color`, `--color`, `--absolute-time`, `--print0`, `--yes`
- [ ] **LOR-P001-071** — Add `--version` with custom long-format output per Standard §13.2 (maintainer footer + project URL)
- [ ] **LOR-P001-072** — Add `--help` long-format footer per Standard §13.2
- [ ] **LOR-P001-073** — Define `Command` enum with all sub-command stubs: `List`, `Show`, `Help`, `Find`, `Search`, `Categories`, `New`, `Update`, `Validate`, `Schema`, `Describe`, `Mcp`
- [ ] **LOR-P001-074** — Each sub-command stub accepts its expected flags (per spec §7) and prints "not yet implemented in Phase 1" with appropriate exit code
- [ ] **LOR-P001-075** — Wire `tracing-subscriber` configured by `--verbose` / `--quiet` flags; stderr only
- [ ] **LOR-P001-076** — Implement `--no-color` / `--color` / `NO_COLOR` env handling for the CLI's own output (TUI rendering comes in Phase 2)

- [ ] **LOR-P001-077** — Make `--json --version` produce SFRS §6 envelope with `metadata.maintainer` and `metadata.website` populated
- [ ] **LOR-P001-078** — Add integration test confirming `loran --version` output matches Standard §13.2 format exactly
- [ ] **LOR-P001-079** — Add integration test confirming `loran --help` footer matches Standard §13.2 format

WP-P1.08 — JSON envelope (`loran-cli` cross-cutting)

Sizing: S | **Critical Path:** Yes (every sub-command needs this) | **Plan §:** 5 | **Deps:** WP-P1.07 | **PRD:** FR-060, FR-061, FR-068

- [ ] **LOR-P001-080** — Define `Envelope<T: Serialize>` struct in `loran-cli/src/envelope.rs` with `metadata` + `data` fields
- [ ] **LOR-P001-081** — Define `Metadata` struct: `tool`, `version`, `command`, `timestamp` (jiff::Timestamp serialised as ISO 8601 UTC + Z), `maintainer`, `website`
- [ ] **LOR-P001-082** — Define `ErrorEnvelope` struct with `error.{code, exit_code, message, hint, timestamp, command, docs_url}` per SFRS §1 Rule 8
- [ ] **LOR-P001-083** — Implement `JsonEmitter` type with `emit_data<T: Serialize>(&self, data: T)`, `emit_error(&self, code, message, hint)`
- [ ] **LOR-P001-084** — Implement custom `serde` serialiser for `jiff::Timestamp` ensuring `Z` suffix (NFR-053)
- [ ] **LOR-P001-085** — Add unit tests round-tripping envelopes through `serde_json` and confirming `Z` suffix on timestamps
- [ ] **LOR-P001-086** — Add unit test confirming error envelopes include all SFRS §1 Rule 8 fields

WP-P1.09 — Agent env-var detection & TTY cascade

Sizing: S | **Critical Path:** No | **Plan §:** 5 | **Deps:** WP-P1.07, WP-P1.08 | **PRD:** FR-061, FR-067

- [ ] **LOR-P001-087** — Add `is-terminal` crate (or use `std::io::IsTerminal`) to `loran-cli`
- [ ] **LOR-P001-088** — Define `OutputMode` enum: `Tui`, `Text`, `Json`
- [ ] **LOR-P001-089** — Implement `detect_output_mode(cli: &Cli) -> OutputMode` checking, in order: explicit `--json` / `--format=json` → `Json`; agent env vars (`AI_AGENT`, `AGENT`, `CI`, `CLAUDECODE`, `CURSOR_AGENT`, `GEMINI_CLI`) → `Json` + stderr warning; non-TTY stdout → `Json`; TTY → `Text` (Phase 1 collapses `Tui` to `Text` until Phase 2 lands the TUI)
- [ ] **LOR-P001-090** — Emit the stderr warning when an agent env var triggers JSON mode (per SFRS §5)
- [ ] **LOR-P001-091** — Add unit tests with env-var injection covering each detection branch

WP-P1.10 — Exit codes + error catalog (`loran-cli`)

Sizing: XS | **Critical Path:** No | **Plan §:** 5 | **Deps:** WP-P1.08 | **PRD:** FR-068, NFR-051

☐ **LOR-P001-092** — Define `ExitCode` enum in `loran-cli/src/exit.rs` with all 12 named codes (0–5 canonical + 6–11 Loran-specific) per spec §9
☐ **LOR-P001-093** — Implement `ExitCode::numeric() -> i32` and `ExitCode::name() -> &'static str`
☐ **LOR-P001-094** — Implement `ExitCode::hint(context: &ErrorContext) -> String` per spec §12.3 (interpolates context like user's query for `NOT_FOUND`)
☐ **LOR-P001-095** — Add unit tests confirming every variant produces a non-empty hint

WP-P1.11 — `loran list` sub-command

Sizing: S | **Critical Path:** No | **Plan §:** 5 | **Deps:** WP-P1.02, WP-P1.07, WP-P1.08 | **PRD:** FR-001

☐ **LOR-P001-096** — Implement `loran list` handler in `loran-cli/src/cmd/list.rs`
☐ **LOR-P001-097** — Wire flags: `--category`, `--replaces`, `--safe-alias-for`, `--fields`
☐ **LOR-P001-098** — Implement filter composition: filters apply in AND, not OR
☐ **LOR-P001-099** — Implement `--fields name,summary` column projection
☐ **LOR-P001-100** — Render text mode as a fixed-width table; render JSON mode as `data: [{...}, ...]` in the envelope
☐ **LOR-P001-101** — Add integration tests for happy text, happy JSON, all four filter flags, and edge cases (empty result, unknown category)

WP-P1.12 — `loran show` sub-command

Sizing: S | **Critical Path:** Yes | **Plan §:** 5 | **Deps:** WP-P1.04, WP-P1.06, WP-P1.07, WP-P1.08 | **PRD:** FR-010, FR-011, FR-012

☐ **LOR-P001-102** — Implement `loran show` handler in `loran-cli/src/cmd/show.rs`
☐ **LOR-P001-103** — Dispatch to `loran-core::resolve_show`; render `IndexHit` via `loran-render::render_text`
☐ **LOR-P001-104** — On `NoEntry`, emit the no-entry diagnostic to stderr in text mode; emit `ErrorEnvelope` with `code = NOT_FOUND = 4` and hint in JSON mode
☐ **LOR-P001-105** — Set `body.kind` to `"custom"` or `"none"` in JSON output (no `"tldr"` in Phase 1)
☐ **LOR-P001-106** — Verify JSON output matches the spec §8 example structure
☐ **LOR-P001-107** — Add integration tests for happy text, happy JSON, no-entry text, no-entry JSON
☐ **LOR-P001-108** — Add `criterion` benchmark confirming cold-cache `loran show eza` completes in <50ms (NFR-001)

WP-P1.13 — `loran help` sub-command

Sizing: S | **Critical Path:** No | **Plan §:** 5 | **Deps:** WP-P1.05, WP-P1.07, WP-P1.08 | **PRD:** FR-020 to FR-025

☐ **LOR-P001-109** — Implement `loran help` handler in `loran-cli/src/cmd/help.rs`

- [ ] **LOR-P001-110** — Dispatch to `loran-core::capture_help`
- [ ] **LOR-P001-111** — Text mode: wrap captured output in monochrome ASCII frame with `LIVE OUTPUT — uncurated, captured from <tool> --help at <ISO 8601 UTC>` header
- [ ] **LOR-P001-112** — Ensure NO Steelbore palette tokens are used in the help frame (visual brand boundary)
- [ ] **LOR-P001-113** — JSON mode: emit envelope with `body.kind = "live_help"`, `body.captured_text`, `body.captured_at`
- [ ] **LOR-P001-114** — On `Timeout`: emit `ExitCode::LIVE_HELP_TIMEOUT = 9` with hint `loran new <tool> --edit`
- [ ] **LOR-P001-115** — Add integration tests with a fixture binary on PATH covering: happy path, retry (-h fallback), timeout, binary-not-found

WP-P1.14 — `loran find` sub-command

Sizing: XS | **Critical Path:** No | **Plan §:** 5 | **Deps:** WP-P1.04, WP-P1.07 | **PRD:** FR-004, FR-005

- [ ] **LOR-P001-116** — Implement `loran find` handler in `loran-cli/src/cmd/find.rs`
- [ ] **LOR-P001-117** — Wire `--safe-alias` flag for strict-mode filtering
- [ ] **LOR-P001-118** — Dispatch to `loran-core::resolve_find`; sort results alphabetically by `name`
- [ ] **LOR-P001-119** — Empty results → "no entries supersede `<legacy>`" diagnostic with hint `loran search <legacy>`
- [ ] **LOR-P001-120** — Add integration tests for broad mode, strict mode, empty result, JSON envelope

WP-P1.15 — `loran search` sub-command

Sizing: XS | **Critical Path:** No | **Plan §:** 5 | **Deps:** WP-P1.04, WP-P1.07 | **PRD:** FR-003

- [ ] **LOR-P001-121** — Implement `loran search` handler in `loran-cli/src/cmd/search.rs`
- [ ] **LOR-P001-122** — Dispatch to `loran-core::resolve_search`; preserve match-score ordering
- [ ] **LOR-P001-123** — Empty result → "no matches for `<query>`" with hint `loran list --category=<...>` if a category-like word was in the query
- [ ] **LOR-P001-124** — Add integration tests covering basic match, multi-word query, empty result, JSON envelope

WP-P1.16 — `loran categories` sub-command

Sizing: XS | **Critical Path:** No | **Plan §:** 5 | **Deps:** WP-P1.02, WP-P1.07 | **PRD:** FR-002

- [ ] **LOR-P001-125** — Implement `loran categories` handler in `loran-cli/src/cmd/categories.rs`
- [ ] **LOR-P001-126** — Load `categories.toml` from the bundled tree; cross-reference with the index's by-category secondary index for counts

- [ ] **LOR-P001-127** — Render text mode as `name | title | count` table; JSON mode as `data: [{name, title, description, count}]`
- [ ] **LOR-P001-128** — Add integration tests for text + JSON output

WP-P1.17 — `loran describe` sub-command

Sizing: XS | **Critical Path:** No | **Plan §:** 5 | **Deps:** WP-P1.07 | **PRD:** FR-063

- [ ] **LOR-P001-129** — Implement `loran describe` handler in `loran-cli/src/cmd/describe.rs`
- [ ] **LOR-P001-130** — Build the describe manifest per SFRS §4: tool name, version, every sub-command with one-line description, capability tags (`read-only`, `network`, `subprocess`)
- [ ] **LOR-P001-131** — Emit as JSON only (describe is always machine-readable per SFRS §4)
- [ ] **LOR-P001-132** — Add integration test validating output against the SFRS describe schema (hand-coded validator in test for now; replaced by schema validation in Phase 3)

WP-P1.18 — `loran schema` placeholder

Sizing: XS | **Critical Path:** No | **Plan §:** 5 | **Deps:** WP-P1.07 | **PRD:** FR-062

- [ ] **LOR-P001-133** — Add `schemars` to `loran-cli/Cargo.toml`
- [ ] **LOR-P001-134** — Derive `JsonSchema` on the `Page` type in `loran-pages` (with `re-export` of the trait)
- [ ] **LOR-P001-135** — Implement `loran schema` handler that emits the schema for `Page` only, with `meta.placeholder: true`
- [ ] **LOR-P001-136** — Document the placeholder status in the sub-command's `--help` text

WP-P1.19 — Phase 1 integration test suite

Sizing: M | **Critical Path:** Yes (Ingot Definition of Done) | **Plan §:** 5 | **Deps:** All preceding Phase 1 WPs | **PRD:** All Phase 1 FRs

- [ ] **LOR-P001-137** — Set up `loran-cli/tests/` with `assert_cmd`, `predicates`, `insta`, `tempfile`
- [ ] **LOR-P001-138** — Add snapshot tests via `insta` for stable text-mode output of every sub-command
- [ ] **LOR-P001-139** — Add JSON-mode validation tests for every sub-command (validate against schemars-derived schemas where available, hand-roll for others)
- [ ] **LOR-P001-140** — Add error-path tests for every named exit code (6 through 9 are reachable in Phase 1)
- [ ] **LOR-P001-141** — Add agent-env-var tests confirming TUI never activates with `AI_AGENT=1`
- [ ] **LOR-P001-142** — Add `criterion` benchmark suite covering `loran show`, `loran list`, `loran search`
- [ ] **LOR-P001-143** — Confirm NFR-001 (<50ms cold) passes for representative known tools
- [ ] **LOR-P001-144** — Confirm NFR-002 (<100ms `loran list` for 1k catalog) passes

☐ **LOR-P001-145** — Wire benchmark suite into CI with regression detection (fail on >10% slowdown)

☐ **LOR-P001-146** — Run Standard §14 compliance checklist; record result in `compliance-log.md`

---

6. Phase 2 — Billet (Stub — Decompose Later)

Status: Work packages defined in `loran-plan-v0_1.md` §6; task-level decomposition deferred until Phase 1 is ready to tag. The following WPs are listed for orientation only — tasks will be added in TODO v0.2.

> **Decompose when:** Phase 1 integration test suite (`LOR-P001-146`) is checked off and Phase 1 compliance audit recorded.

WP-P2.01 — Postcard index cache

Sizing: S | **Critical Path:** Yes | **Plan §:** 6 | **PRD:** NFR-001 *Task decomposition deferred to TODO v0.2.*

WP-P2.02 — TUI shell (`loran-tui`)

Sizing: M | **Critical Path:** No | **Plan §:** 6 | **PRD:** FR-006 *Task decomposition deferred to TODO v0.2.*

WP-P2.03 — TUI browse view

Sizing: M | **Critical Path:** No | **Plan §:** 6 | **PRD:** FR-006 *Task decomposition deferred to TODO v0.2.*

WP-P2.04 — TUI detail view

Sizing: M | **Critical Path:** No | **Plan §:** 6 | **PRD:** FR-013, FR-014, FR-015 *Task decomposition deferred to TODO v0.2.*

WP-P2.05 — TUI fuzzy search overlay

Sizing: S | **Critical Path:** No | **Plan §:** 6 | **PRD:** FR-003 *Task decomposition deferred to TODO v0.2.*

WP-P2.06 — TUI in-app help

Sizing: XS | **Critical Path:** No | **Plan §:** 6 | **PRD:** NFR-062 *Task decomposition deferred to TODO v0.2.*

WP-P2.07 — HTTP client + manifest fetch

Sizing: S | **Critical Path:** Yes | **Plan §:** 6 | **PRD:** FR-040, FR-041 *Task decomposition deferred to TODO v0.2.*

WP-P2.08 — Tar/gzip extraction (atomic)

Sizing: S | **Critical Path:** Yes | **Plan §:** 6 | **PRD:** FR-045, NFR-020 *Task decomposition deferred to TODO v0.2.*

WP-P2.09 — Minisign verification

Sizing: S | **Critical Path:** Yes | **Plan §:** 6 | **PRD:** FR-043, FR-044, NFR-030, NFR-031 *Task decomposition deferred to TODO v0.2.*

WP-P2.10 — Upstream pages tarball pipeline (client side)

Sizing: S | **Critical Path:** Yes | **Plan §:** 6 | **PRD:** FR-040 to FR-046 *Task decomposition deferred to TODO v0.2.*

WP-P2.11 — tldr-pages tarball fetch

Sizing: XS | **Critical Path:** No | **Plan §:** 6 | **PRD:** FR-047, FR-048 *Task decomposition deferred to TODO v0.2.*

WP-P2.12 — `loran update` sub-command wiring

Sizing: XS | **Critical Path:** No | **Plan §:** 6 | **PRD:** FR-040 to FR-049 *Task decomposition deferred to TODO v0.2.*

WP-P2.13 — Overlay merge engine

Sizing: M | **Critical Path:** No (parallel) | **Plan §:** 6 | **PRD:** FR-050 to FR-054 *Task decomposition deferred to TODO v0.2.*

WP-P2.14 — Page template + `loran new` (non-interactive)

Sizing: S | **Critical Path:** No | **Plan §:** 6 | **PRD:** FR-030 to FR-032, FR-034 *Task decomposition deferred to TODO v0.2.*

WP-P2.15 — `loran new` interactive mode

Sizing: M | **Critical Path:** No | **Plan §:** 6 | **PRD:** FR-033 *Task decomposition deferred to TODO v0.2.*

WP-P2.16 — `loran validate` sub-command

Sizing: S | **Critical Path:** No | **Plan §:** 6 | **PRD:** FR-035, FR-036 *Task decomposition deferred to TODO v0.2.*

WP-P2.17 — tldr fallback in resolution chain

Sizing: S | **Critical Path:** No | **Plan §:** 6 | **PRD:** FR-011 *Task decomposition deferred to TODO v0.2.*

WP-P2.18 — Phase 2 integration test suite

Sizing: M | **Critical Path:** Yes (Billet Definition of Done) | **Plan §:** 6 | **PRD:** All Phase 2 FRs *Task decomposition deferred to TODO v0.2.*

7. Phase 3 — Bloom (Stub — Decompose Later)

Status: Work packages defined in `loran-plan-v0_1.md` §7; task-level decomposition deferred until Phase 2 is ready to tag.

> **Decompose when:** Phase 2 integration test suite is checked off and Phase 2 compliance audit recorded.

WP-P3.01 — Full JSON Schema emission

Sizing: M | **Critical Path:** Yes | **Plan §:** 7 | **PRD:** FR-062 *Task decomposition deferred to TODO v0.3.*

WP-P3.02 — MCP server crate (`loran-mcp`)

Sizing: M | **Critical Path:** Yes | **Plan §:** 7 | **PRD:** FR-064, FR-065 *Task decomposition deferred to TODO v0.3.*

WP-P3.03 — `loran mcp` sub-command wiring

Sizing: XS | **Critical Path:** Yes | **Plan §:** 7 | **PRD:** FR-064 *Task decomposition deferred to TODO v0.3.*

WP-P3.04 — `DescribeIngestor` implementation

Sizing: M | **Critical Path:** No (parallel) | **Plan §:** 7 | **PRD:** FR-071, FR-072, FR-073 *Task decomposition deferred to TODO v0.3.*

WP-P3.05 — Minisign key rotation documentation

Sizing: S | **Critical Path:** No | **Plan §:** 7 | **PRD:** Open Question 1 *Task decomposition deferred to TODO v0.3.*

WP-P3.06 — Cross-distro overlay surfacing

Sizing: S | **Critical Path:** No | **Plan §:** 7 | **PRD:** §13.1 *Task decomposition deferred to TODO v0.3.*

WP-P3.07 — Phase 3 integration test suite

Sizing: M | **Critical Path:** Yes (Bloom Definition of Done) | **Plan §:** 7 | **PRD:** All Phase 3 FRs *Task decomposition deferred to TODO v0.3.*

---

8. Cross-Cutting Workstreams

These workstreams run alongside the phase work. Initial tasks land in Phase 0 (where they bootstrap); ongoing maintenance is performed within whichever WP is currently active.

Workstream	Initial-task location	Plan §
WP-CC.01 CI pipeline maintenance	WP-P0.05 (LOR-P000-032 to LOR-P000-041)	8
WP-CC.02 Benchmark suite	WP-P1.19 (LOR-P001-142 to LOR-P001-145)	8
WP-CC.03 Documentation maintenance	WP-P0.02 + WP-P0.03 (ongoing)	8
WP-CC.04 Page authoring (content)	WP-P1.03 (LOR-P001-028 to LOR-P001-035) + ongoing	8
WP-CC.05 Security review	Pre-release per-phase	8
WP-CC.06 Release engineering	Defined in Plan §10; first release after Ingot	10
WP-CC.07 Publisher pipeline coordination	Phase 2 onwards (separate project)	8
WP-CC.08 Compliance audits	LOR-P001-146 + per-phase + per-release	8

When a cross-cutting task arises outside an active WP (e.g., a CVE in a dependency requiring an audit pass), record it as `LOR-PCC-NNN` and resolve before the next release.

---

9. Document Revision Strategy

This TODO is a living document. Versioning policy:

Trigger	Action
Phase 1 (`LOR-P001-146`) checked off	Revise to v0.2: Phase 2 decomposed, Phase 1 archived
Phase 2 integration test suite checked off	Revise to v0.3: Phase 3 decomposed, Phase 2 archived
Phase 3 integration test suite checked off	Revise to v1.0 of TODO; archive to historical record
Spec or PRD revision changes a requirement	Revise this TODO synchronously; affected tasks tagged
New cross-cutting concern emerges	Add `LOR-PCC-NNN` tasks inline; record in revision

Archiving discipline. Completed phases are not deleted from this document — they remain as a historical record with all tasks checked off. New revisions add (don't replace) phase sections. Once a phase is complete and tagged, its tasks are not modified, even if errors are later found (those become hotfix tasks in the current revision).

Task ID stability. Task IDs are immutable once assigned. If a task is determined to be unnecessary or obsolete, leave it in the document with strikethrough and a note (`~~LOR-P001-XXX~~ — retired: superseded by LOR-P001-YYY`). Do not reuse retired IDs.

Forged in Steelbore.